Interview Q&A Cheat Sheet

Likely questions, organized by topic, with answers you can say out loud. Each answer is deliberately short — expand with numbers from the other files if the interviewer wants more depth.

"Walk me through this project."

See `00_overview.md` for the 30-second version. If they want more, walk through it in pipeline order: clean -> explore -> engineer features -> train three models -> evaluate properly -> deploy behind an API.

"Why this dataset?"

It's a real, publicly available dataset (UCI, 30,000 Taiwanese credit card customers) with a genuinely imbalanced, realistic target — not a toy problem. It let me practice the full pipeline end to end: data quality issues, imbalance handling, model comparison, and deployment.

"What was wrong with the raw data, and how did you fix it?"

`EDUCATION` is documented as 1–4, but the raw data also has 0, 5, 6. `MARRIAGE` is documented as 1–3, but the raw data also has 0. These are undocumented codes, almost certainly data-entry artifacts. I folded them into the existing "other" category rather than dropping those rows — that keeps a few hundred otherwise-valid rows instead of throwing them away over one bad column.

"Why these two engineered features specifically?"

Because the EDA justified them directly. The utilization-ratio histogram shows defaulters cluster near full credit usage; the delayed-payments bar chart shows default rate climbing from 12% to 70% as delayed months go from 0 to 6, and it's the single highest-correlation feature in the dataset (0.40). I didn't guess at features — I looked at the data first, then built features around what the data actually showed.

"Why three different models?"

Different models make different assumptions about the data (linear vs. non-linear decision boundary, single estimator vs. ensemble), and you can't know which fits best without trying a spread and measuring. See `02_models_explained.md` for how each one works and why Random Forest won.

"Why did Random Forest win?"

It captures the same non-linear patterns a single Decision Tree can, but averaging 400 trees trained on random subsets of the data cancels out most of any single tree's overfitting — better generalization to unseen data. That's reflected directly in the numbers: best accuracy, precision, recall, *and* F1 of the three.

"How did you choose hyperparameters?"

`GridSearchCV` with 5-fold cross-validation over a small, deliberately readable grid per model (e.g. Random Forest: `n_estimators` in [200, 400], `max_depth` in [6, 10, None]). It fits every combination on 5 different train/validation splits and picks whichever generalized best, scored on F1.

"Why F1 and not accuracy to pick the winner?"

Because the classes are imbalanced (22% defaulters) — a model that always predicts "no default" would already score ~78% accuracy while being useless. F1 balances precision and recall, which actually measures whether the model is catching real defaulters, not just defaulting to the majority class. Full derivation with real numbers in `03_metrics_explained.md`.

"How did you handle the class imbalance?"

`class_weight="balanced"` on every model, which penalizes mistakes on the minority (default) class roughly proportional to how rare it is, instead of treating every mistake equally. I considered SMOTE/resampling too, but `class_weight` achieves a similar effect without synthesizing or discarding rows, and it's simpler to reason about for a moderate (not extreme) imbalance like this one.

"What does your confusion matrix tell you?"

For the winning model: 807 true positives, 520 false negatives, 833 false positives, 3,840 true negatives. In business terms: it catches 61% of actual defaulters (recall), and just under half of what it flags as "high risk" actually defaults (precision). The 520 missed defaulters are the number I'd focus on improving first, since missing a real defaulter is usually costlier to a lender than one extra false alarm.

"If you had more time, what would you improve?"

Three honest answers, in order of effort: 1. **Tune the decision threshold** below the default 0.5 to trade some precision for more recall, since missing a defaulter is costlier than a false alarm — I didn't implement this to keep the project focused, but it's a natural next step (`model.predict_proba()` already gives the raw probability; this is just where you draw the line). 2. **Add explainability** (e.g. SHAP) so each individual prediction comes with "why" — useful for a human reviewer, and often required in regulated lending. 3. **Fairness audit** — check whether the model's error rates differ meaningfully across `SEX`, `AGE`, or `EDUCATION` groups before using it for anything with real consequences.

"Why FastAPI and Pydantic for deployment?"

FastAPI gives async request handling and auto-generated interactive docs (`/docs`) with very little boilerplate. Pydantic schemas validate every field's type and range (e.g. `age` between 18–100, `education` in 1–4) *before* the request ever reaches the model — bad input gets a clean 422 error instead of crashing the model or silently producing a garbage prediction.

"How would you know if this model is actually good enough to deploy?"

Honestly: this specific project is a learning/portfolio exercise, not a production system, so I'd be explicit about that distinction in a real interview. For actual production use I'd want: a fairness audit across protected attributes, a cost-based (not just F1-based) threshold chosen with the actual business's cost of a false negative vs. false positive, and monitoring for data drift once deployed, since repayment behavior patterns from 2005 Taiwan won't necessarily hold elsewhere or later.